Wittmann HR Portal

Full technical and user manual for HR/admin management portal

Project folder: E:\Wittmann Portel

Generated: 16-07-2026 05:48 PM

1. Purpose and Scope

This portal is the HR/admin side of the Wittmann interview platform. It is separate from the candidate interview system but uses the same PostgreSQL database. HR users can review candidates, open reports and resumes inside the website, download documents, approve shortlisted candidates, manage roles, manage interview timings, manage the live question bank, import questions from Excel/CSV, configure SMTP, and manage HR portal users.

2. Main Features

• HR login with admin and restricted-user permissions.

• Dashboard with interview/candidate statistics.

• Candidate list grouped by automatic/manual shortlist status with marks and reviewer notes.

• Open report and resume inside the site, plus separate download options.

• Approve individual or selected candidates and send second-round email when SMTP is configured.

• Role management with allowed degrees, allowed question sets, aptitude time, and programming time.

• Interview timings screen for changing role-specific aptitude/programming minutes used by the candidate system.

• Question bank CRUD with active/inactive toggle and role/question-set filters.

• Excel/CSV import for bulk question upload.

• Recycle bin with soft-delete restore support for users, roles, questions, and candidates.

• SMTP settings screen for email delivery configuration.

• Same database as candidate app, so changes appear in live interviews without code edits.

3. Technology Stack and Libraries

Runtime: Python Flask HR portal with PostgreSQL, server-rendered HTML templates, CSS, Excel/CSV import, PDF/document inline viewing and downloads.

Pinned Python Libraries from requirements.txt

• Flask==3.0.3

• python-dotenv==1.0.1

• psycopg[binary]==3.2.13

• Werkzeug==3.0.3

• openpyxl==3.1.5

• pypdf==4.3.1

• python-docx==1.1.2

Imported Libraries and Modules

• from datetime import timedelta

• from dotenv import load_dotenv

• from email.message import EmailMessage

• from flask import Flask, flash, redirect, render_template, request, send_file, session, url_for

• from functools import lru_cache

• from functools import wraps

• from pathlib import Path

• from psycopg.rows import dict_row

• from psycopg.types.json import Jsonb

• from urllib.parse import urlsplit, urlunsplit

- from werkzeug.security import check_password_hash, generate_password_hash
- from werkzeug.utils import secure_filename
- import csv
- import json
- import os
- import psycopg
- import re
- import secrets
- import smtplib

Important Library Usage

- Flask: login/session handling, routes, flash messages, templates, file streaming, and downloads.
- psycopg: shared PostgreSQL data access using dict rows and JSONB snapshots.
- Werkzeug: password hashing/checking and secure uploaded filenames.
- openpyxl: Excel question-bank import.
- pypdf and python-docx: document text/extraction/viewing support where applicable.
- python-dotenv: local .env configuration loading.
- csv/json/re/secrets/smtplib: imports, data cleaning, secure password/token generation, and email delivery.

4. Folder Structure

The following tree excludes runtime-heavy folders such as .git, .venv, cache folders, logs, and Python cache files.

```
Wittmann Portel/
.env
.env.example
.gitignore
app.py
pyproject.toml
README.md
requirements.txt
vercel.json
static/
css/style.css
images/robot_automation_teaser_2020_11_web.jpg
images/wittmann-logo-hq.png
templates/
artifact_view.html
base.html
candidates.html
dashboard.html
database_error.html
interview_timings.html
login.html
question_form.html
question_import.html
questions.html
recycle_bin.html
role_form.html
roles.html
smtp_settings.html
user_form.html
users.html
```

5. Source File Responsibilities

File or folder	Responsibility
app.py	Complete Flask HR portal: login, permissions, dashboard, candidates, approvals, roles, users, SMTP settings, questions, imports, timings, recycle bin, report/resume view and download routes.
templates/base.html	Common layout/navigation, access-aware UI shell, flash messages, page structure.
templates/login.html	HR portal login screen.
templates/dashboard.html	Management dashboard statistics and overview.

File or folder	Responsibility
templates/candidates.html	Candidate review list, shortlist sections, approvals, report/resume actions, reviewer notes.
templates/artifact_view.html	Inline website viewer for PDF reports and resumes with download option.
templates/roles.html / role_form.html	Role listing and editor including degrees, question sets, aptitude/programming timings.
templates/interview_timings.html	Dedicated time-setting page for aptitude and programming rounds.
templates/questions.html / question_form.html / question_import.html	Question bank management and Excel/CSV import screens.
templates/users.html / user_form.html	HR user creation, permissions, activation, deletion, password reset.
templates/smtp_settings.html	SMTP configuration interface.
templates/recycle_bin.html	Soft-delete recovery interface.
static/css/style.css	Portal visual design, tables, forms, dashboard cards, responsive behavior.
vercel.json / pyproject.toml	Vercel deployment configuration for the HR portal.

6. Web Pages and API Routes

Route	Methods	Function
/	GET, POST	login
/logout	GET	logout
/dashboard	GET	dashboard
/candidates	GET	candidates
/roles	GET	roles
/roles/new	GET, POST	new_role
/roles/<int:role_id>/edit	GET, POST	edit_role
/users	GET	users
/users/new	GET, POST	new_user
/settings/smtp	GET, POST	smtp_settings
/questions	GET	questions
/interview-timings	GET, POST	interview_timings
/questions/new	GET, POST	new_question
/questions/import	GET, POST	import_questions
/questions/<int:question_id>/edit	GET, POST	edit_question
/recycle-bin	GET	recycle_bin
/download/report/<interview_id>	GET	download_report
/view/report/<interview_id>	GET	view_report
/inline/report/<interview_id>	GET	inline_report
/download/resume/<interview_id>	GET	download_resume
/view/resume/<interview_id>	GET	view_resume
/inline/resume/<interview_id>	GET	inline_resume

7. HR User Workflow Manual

• Step 1: HR opens the portal URL and logs in with an active HR account.

• Step 2: Dashboard gives high-level totals and shortlist status information.

• Step 3: Candidates page shows interview results. HR can filter/search, open reports/resumes inside the site, download files, add reviewer notes, or approve candidates.

• Step 4: Roles page is used to create or edit roles. Allowed degree and question-set settings control which candidates and questions are eligible.

• Step 5: Interview Timings page sets aptitude and programming minutes for each role. Candidate interview system reads these values when generating the interview page.

• Step 6: Question Bank page lets HR add, edit, enable, disable, and filter questions by role, section, active status, and question set.

• Step 7: Import Excel page allows bulk upload of questions and roles using the documented columns.

• Step 8: Users page lets the main admin create HR accounts and assign tab permissions.

• Step 9: SMTP Settings page configures email delivery for HR user access and candidate approval emails.

• Step 10: Recycle Bin allows recently deleted records to be restored before purge.

8. Database and Shared Data Model

The portal works on the same PostgreSQL database used by the candidate application. This is why role changes, question changes, and timing changes can affect new interviews without restarting or editing the candidate app.

• hr_portal_users: HR login accounts, password hashes, admin flag, tab permissions, active state, delete metadata.

• roles: role definitions, allowed_degrees, allowed_question_sets, aptitude_minutes, programming_minutes, soft-delete metadata.

• question_bank: live editable question bank used by candidate app for new assigned papers.

• interviews: completed/interview records, total score, report path, status, shortlist details, reviewer notes.

• users: candidate identity and resume path.

• candidate_answers and interview_questions: details used when viewing candidate reports and answer history.

• recycle_bin: snapshots for restore of soft-deleted portal records.

9. Question Import Format

Excel/CSV import accepts .xlsx, .xlsm, and .csv. Important columns are:

```
Role Name, Role Slug, Question Code, Section, Topic, Difficulty, Question, Options, Correct Answer, Keywords,
Marks, Active
```

• Required columns: Role Name, Section, Topic, Question, Correct Answer.

• Options and keywords can be one per line or separated with |.

• If Question Code already exists, importing updates that question instead of creating a duplicate.

• A role should have enough active questions for the candidate app: 30 Aptitude and 5 Programming for the requested current behavior.

10. Configuration and Environment

• DATABASE_URL: same PostgreSQL connection as the candidate interview app.

• INTERVIEW_PROJECT_DIR: local path to E:\wittmann_interview_ai for report/resume resolution.

• REPORT_DIR: report directory in the candidate project.

• HR_PORTAL_URL: public/local URL used in emails.

• SHORTLIST_MIN_SCORE: cutoff for automatic shortlisting views.

• HR_USERNAME, HR_PASSWORD, HR_EMAIL: initial admin account settings.

• HR_PASSWORD_HASH: recommended production replacement for plain HR_PASSWORD.

• SMTP_HOST, SMTP_PORT, SMTP_USER, SMTP_PASSWORD, SMTP_FROM: email delivery settings.

• SECRET_KEY: Flask session security key.

11. Running Locally

```
cd "E:\Wittmann Portel"
python -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
python app.py
```

Open http://localhost:5050. The portal is HR/admin only; it does not contain candidate registration, OTP, camera check, or interview routes.

12. Deployment

• Configured for Vercel using pyproject.toml and vercel.json.

• Production DATABASE_URL must point to hosted PostgreSQL; localhost will not work on Vercel.

• Set SECRET_KEY, DATABASE_URL, HR_USERNAME, HR_PASSWORD or HR_PASSWORD_HASH, HR_EMAIL, and HR_PORTAL_URL as deployment environment variables.

• For file viewing to work in production, report/resume paths must be reachable from the portal host. If portal and candidate app are hosted separately, use shared storage or store files where both services can read them.

13. Operational Notes and Troubleshooting

• If reports/resumes do not open, verify INTERVIEW_PROJECT_DIR and REPORT_DIR and confirm files exist on the host.

• If email is not sent, configure SMTP settings and use an app password for Gmail/Google Workspace.

• If a user cannot see a tab, check tab_permissions or admin status in the Users page.

• If a candidate paper does not reflect new questions, remember already-started interviews use the snapshot saved in interview_questions; new interviews use latest active question_bank records.

• If role timings do not change for a current candidate, the candidate may need to start a new interview session because timings are passed when the interview page is loaded.